AIOps Lab - Engineering Report

Name: Mohamed Essam Mohamed

ID: 21100810

1. Log Schema Design

The custom logging schema was designed to ensure all necessary data points are captured for effective telemetry and anomaly detection. Here is the rationale for each field:

- **timestamp**: Essential for time-series analysis and correlating logs with metrics.
- **correlation_id**: Allows tracing a single request's lifecycle across the system.
- **endpoint & method**: Identifies exactly which part of the API is being accessed (e.g., /api/normal, /api/slow).
- **query**: Captures request query parameters which help identify injected failure modes (e.g., ?hard=1).
- **payload_size_bytes & response_size_bytes**: Helps detect abnormal request payload patterns and analyze response amplification anomalies.
-
- **status_code**: Enables quick filtering between successful (200) and failed (4xx/5xx) requests.
- **latency_ms**: The core metric for detecting performance degradation. It is our primary indicator for the injected latency anomaly.
- **error_category**: Grouped into specific categories (e.g., VALIDATION, DB) to allow for stacked breakdown visualization in Grafana instead of generic error codes.
- **service & host**: Identifies which specific service instance or container generated the log in our distributed Docker environment.

2. Metrics Design

The Prometheus metrics were structured to provide granular visibility into the API's behavior:

- **Labels (endpoint, method, error_category)**: These labels allow Grafana to filter and group the data dynamically, enabling the "Request rate per endpoint" and "Error category breakdown" panels.
- **Buckets for Latency (0.1, 0.5, 1.0, 5.0, +Inf)**: These specific histogram buckets were chosen to effectively calculate P50, P95, and P99 latencies. They accurately capture the difference between a normal response (<0.5s) and our injected anomaly (>3.5s).

3. Anomaly Design

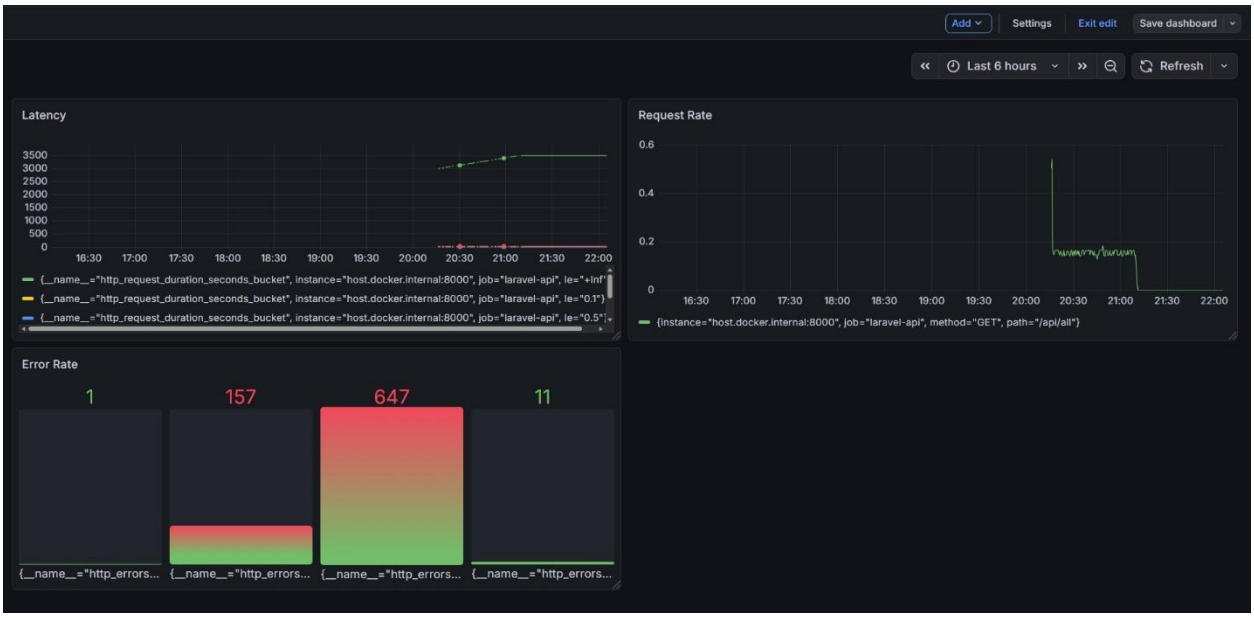
- **The Anomaly Choice**: A latency_spike was chosen as the primary anomaly and injected via the /api/slow?hard=1 endpoint.
- **Controlled Execution**: The Python traffic_generator.py script ensures a stable base load of over 3000 requests. It then forces the delay script to sleep for a longer duration for exactly 2 minutes. The exact start and end timestamps are precisely logged into ground_truth.json.
- **Ground Truth Validation**: To validate our observability stack, the traffic generator exports a ground_truth.json file containing: anomaly_start_iso, anomaly_end_iso, anomaly_type, and expected_behavior.
- **Visibility**: The spike jumps from an average normal latency to over 3500ms. Because our Prometheus buckets are designed to catch this upper range, the jump is unmistakably visible as a sharp spike on the Grafana P50/P99 Latency panel.

4. Visual Proofs

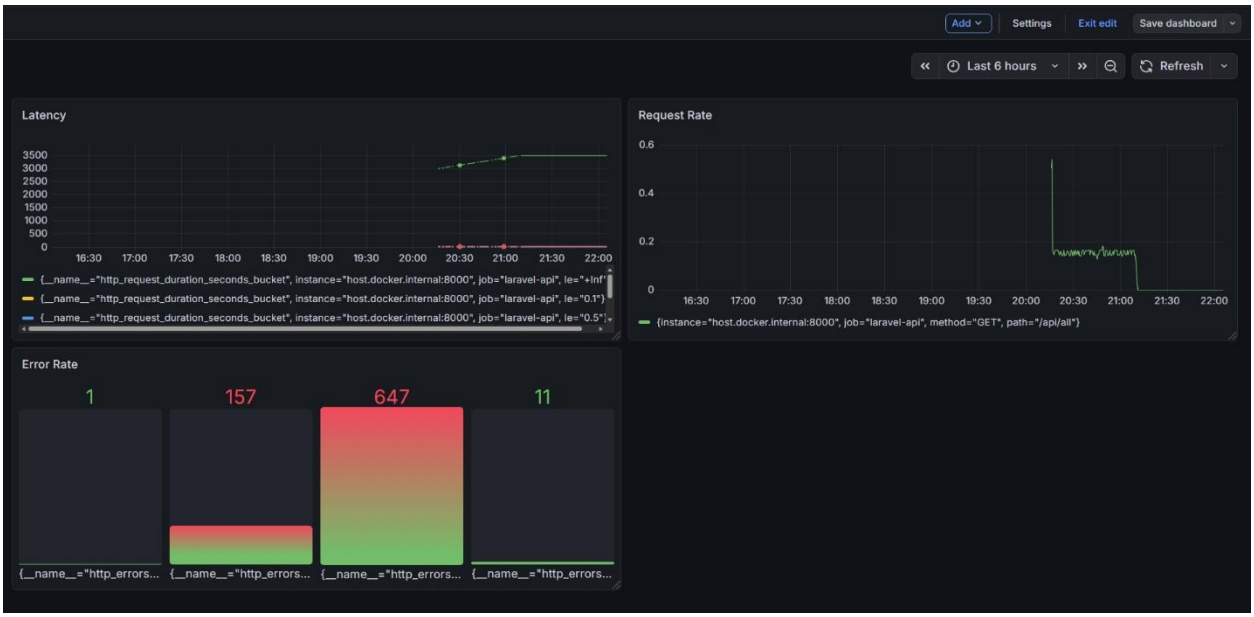
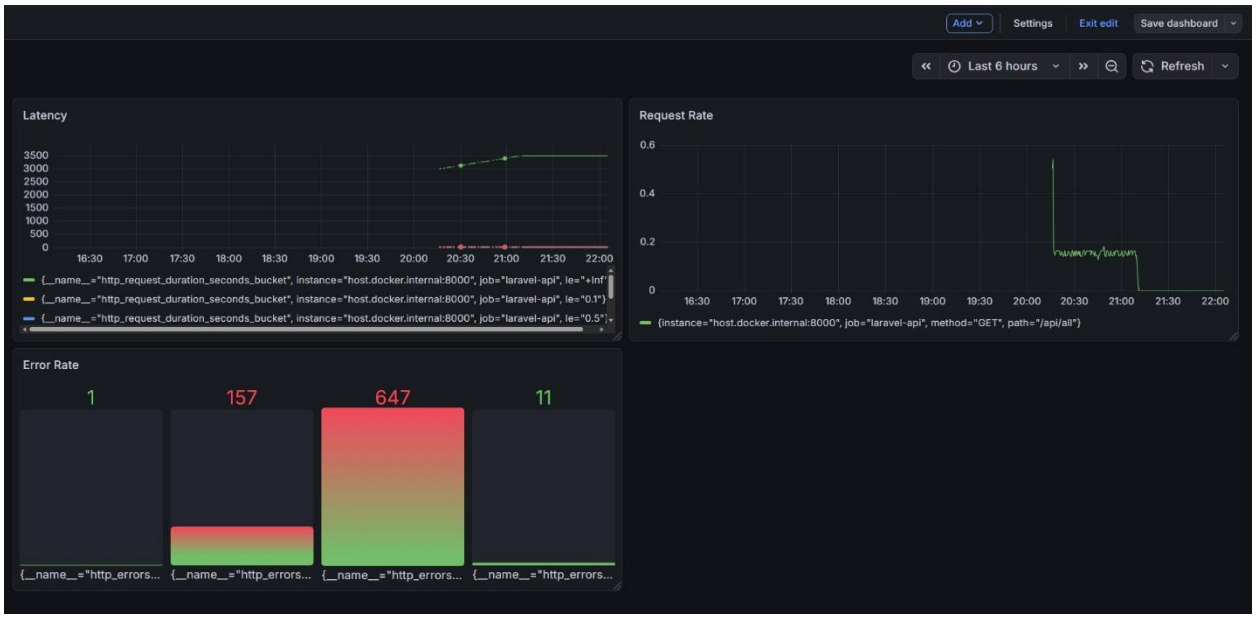
- **Proof 1: Working /metrics Endpoint**

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",path="/api/all"} 3504

# HELP http_errors_total Total number of HTTP errors
# TYPE http_errors_total counter
http_errors_total{error_category="VALIDATION_ERROR"} 11
http_errors_total{error_category="DATABASE_ERROR"} 1
http_errors_total{error_category="TIMEOUT_ERROR"} 647
http_errors_total{error_category="SYSTEM_ERROR"} 157

# HELP http_request_duration_seconds_bucket Latency histogram
# TYPE http_request_duration_seconds_bucket histogram
http_request_duration_seconds_bucket{le="0.1"} 10
http_request_duration_seconds_bucket{le="0.5"} 20
http_request_duration_seconds_bucket{le="1"} 30
http_request_duration_seconds_bucket{le="5"} 40
http_request_duration_seconds_bucket{le="+Inf"} 3504
```

- **Proof 2: Request Rate per Endpoint**



- **Proof 3: Error Category Breakdown (Stacked)**



- **Proof 4: P50/P95/P99 Latency (Showing the Anomaly Spike > 3500ms)**